

AHDS - Analyse pratique de circuits avec MATLAB #02

Commande PWM de LED avec NE555 - comparaison BJT / N-MOSFET

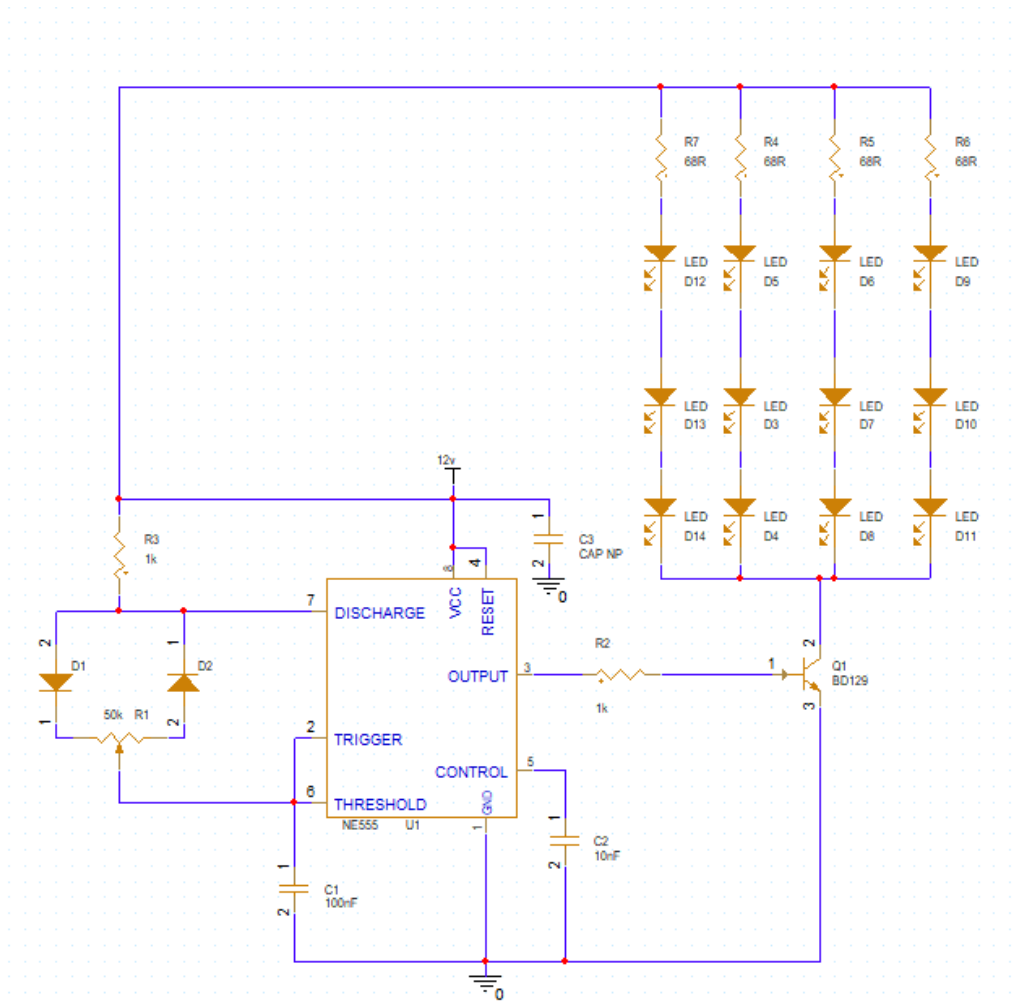
Étude théorique, calculs, simulation et graphiques MATLAB

1. Objectif

Ce document étudie un variateur de luminosité à modulation de largeur d'impulsion (PWM) basé sur un NE555. L'objectif principal n'est pas de fabriquer une lampe finale, mais d'utiliser une application électronique simple pour montrer comment MATLAB peut servir à calculer, simuler et visualiser le fonctionnement d'un circuit.

La première partie considère une commutation par transistor bipolaire (BJT). La seconde compare cette solution à une commutation par MOSFET canal N lorsque le courant de charge augmente. Les modèles MATLAB proposés sont volontairement pédagogiques : ils permettent de modifier facilement les paramètres et d'observer leurs effets.

2. Schéma étudié



Dans le schéma, les diodes autour du potentiomètre séparent les chemins de charge et de décharge du condensateur de temporisation. Cette disposition permet de modifier le rapport cyclique tout en maintenant une fréquence relativement stable. La sortie du NE555 commande ensuite l'étage de puissance qui alimente les branches de LED.

3. Paramètres de départ pour l'étude

Paramètre	Valeur d'étude	Rôle
VCC	12 V	Alimentation du NE555 et de la charge
R1	50 kΩ	Potentiomètre de réglage PWM
R3	1 kΩ	Résistance minimale dans le réseau de temporisation
C1	100 nF	Condensateur de temporisation
R2	1 kΩ	Résistance entre la sortie du NE555 et la base du BJT
C2	10 nF	Filtrage de la broche CONTROL
C3	100 nF recommandé	Découplage local du NE555
RLED	68 Ω (valeur du schéma)	Résistance série de chaque branche LED
Charge	4 branches × 3 LED	Charge utilisée pour l'exemple initial

Remarque : la valeur de 68 Ω doit être interprétée en fonction de la tension directe et du courant nominal des LED réellement choisies. Dans ce document, elle est conservée comme paramètre du schéma et peut être modifiée dans MATLAB.

4. Principe du PWM

Le PWM ne réduit pas directement l'amplitude de la tension d'alimentation. La charge est commutée rapidement entre l'état ON et l'état OFF. La luminosité moyenne dépend principalement du rapport entre le temps de conduction et la période totale.

$$D = t_{ON} / T$$

$$T = t_{ON} + t_{OFF}$$

$$f_{PWM} = 1 / T$$

Pour une charge dont le courant à l'état ON vaut I_{ON} , le courant moyen idéal peut être approximé par :

$$I_{AVG} \approx D \times I_{ON}$$

De même, pour une charge commutée idéale, la puissance moyenne peut être approximée par :

$$P_{AVG} \approx D \times P_{ON}$$

5. Temporisation du NE555

Dans un NE555 astable, le condensateur de temporisation évolue entre environ $1/3 V_{CC}$ et $2/3 V_{CC}$. Les diodes du réseau de potentiomètre permettent d'utiliser des résistances différentes pendant la charge et la décharge.

$$V_{C,bas} \approx V_{CC} / 3$$

$$V_{C,haut} \approx 2V_{CC} / 3$$

Pour une approximation pédagogique, si R_{CH} est la résistance équivalente du chemin de charge et R_{DIS} celle du chemin de décharge :

$$t_{ON} \approx 0,693 \times R_{CH} \times C1$$

$$t_{OFF} \approx 0,693 \times R_{DIS} \times C1$$

$$D \approx R_{CH} / (R_{CH} + R_{DIS})$$

Dans le modèle MATLAB, la position du potentiomètre peut être représentée par une variable comprise entre 0 et 1. Les résistances de charge et de décharge sont alors calculées à partir de cette position et de la résistance minimale $R3$.

6. Courant des branches LED

Chaque branche comporte trois LED en série et une résistance. Lorsque le transistor de commutation est passant, le courant d'une branche peut être estimé par :

$$I_{LED,ON} \approx (V_{CC} - 3V_F - V_{SW}) / R_{LED}$$

V_{SW} représente la chute de tension de l'élément de commutation : $V_{CE(sat)}$ pour un BJT ou approximativement $I_D \times R_{DS(on)}$ pour un MOSFET.

Avec quatre branches identiques :

$$I_{TOTAL,ON} \approx 4 \times I_{LED,ON}$$

$$I_{TOTAL,AVG} \approx D \times I_{TOTAL,ON}$$

7. Étude de $R2 = 1 \text{ k}\Omega$ avec le BJT

La résistance $R2$ limite le courant fourni par la sortie du NE555 vers la base du transistor. Pour une étude simple, avec une sortie haute proche de 11 V et $V_{BE} \approx 0,8 \text{ V}$:

$$I_B \approx (V_{OUT} - V_{BE}) / R2$$

$$I_B \approx (11 - 0,8) / 1000 \approx 10,2 \text{ mA}$$

Cette valeur est beaucoup plus raisonnable que celle obtenue avec 120Ω . Pour une réalisation réelle, la valeur finale de $R2$ doit néanmoins être vérifiée en fonction du courant collecteur et du transistor choisi afin de garantir une commutation correcte.

8. Dissipation du BJT

Lorsque le BJT fonctionne comme interrupteur, une première approximation des pertes en conduction est :

$$P_{BJT} \approx V_{CE(sat)} \times I_C \times D$$

Les pertes augmentent donc presque linéairement avec le courant de charge et le rapport cyclique. À courant élevé, elles peuvent devenir suffisamment importantes pour nécessiter une vérification thermique et éventuellement un dissipateur.

9. Remplacement par un MOSFET canal N

Pour une charge plus importante, un MOSFET canal N logique peut être utilisé comme interrupteur low-side. Contrairement au BJT, sa perte de conduction dépend principalement de $R_{DS(on)}$.

$$P_{MOSFET,cond} \approx I_D^2 \times R_{DS(on)} \times D$$

À basse fréquence PWM, les pertes de commutation peuvent rester modestes, mais elles ne sont pas nulles. Une analyse plus complète peut ajouter les pertes de commutation :

$$PSW \approx 0,5 \times V_{DS} \times I_D \times (t_r + t_f) \times f_{PWM}$$

$$P_{MOSFET, total} \approx P_{MOSFET, cond} + PSW$$

Le choix du MOSFET ne doit donc pas être basé uniquement sur son courant maximal annoncé. Il faut également vérifier $R_{DS(on)}$ à la tension de grille réellement disponible, la puissance dissipable, le boîtier, la température de jonction et la capacité de la source d'alimentation.

10. Exemple d'augmentation du nombre de LED

Pour illustrer l'intérêt du MOSFET, MATLAB peut faire varier le nombre total de LED. Avec une alimentation de 12 V et trois LED par branche, 300 LED correspondent à 100 branches.

$$N_{branches} = N_{LED} / 3$$

Si chaque branche consomme 20 mA à l'état ON :

$$I_{TOTAL, ON} = 100 \times 20 \text{ mA} = 2 \text{ A}$$

Avec un MOSFET dont $R_{DS(on)} = 20 \text{ m}\Omega$, à $D = 100 \%$:

$$P_{MOSFET, cond} = 2^2 \times 0,020 = 0,080 \text{ W}$$

Cet exemple montre pourquoi un MOSFET correctement choisi peut réduire fortement les pertes. Il ne signifie toutefois pas qu'un nombre donné de LED est automatiquement acceptable : l'alimentation, le routage, les résistances de branche, la température et les caractéristiques des LED doivent également être dimensionnés.

11. Graphiques MATLAB proposés

- Tension PWM de sortie pour plusieurs rapports cycliques : 10 %, 25 %, 50 %, 75 % et 90 %.
- Tension du condensateur C1 entre environ 1/3 VCC et 2/3 VCC.
- Courant instantané d'une branche LED $I_{LED}(t)$.
- Courant total de la charge $I_{TOTAL}(t)$.
- Courant moyen des LED en fonction du rapport cyclique.
- Puissance moyenne de la charge en fonction du rapport cyclique.
- Rapport cyclique en fonction de la position du potentiomètre.
- Pertes du BJT en fonction du courant et du rapport cyclique.
- Pertes de conduction du MOSFET en fonction du courant et du rapport cyclique.
- Comparaison directe PBJT / PMOSFET.
- Courant total en fonction du nombre de LED.
- Pertes de puissance en fonction du nombre de LED pour BJT et MOSFET.

12. Programme MATLAB

Le script suivant constitue une simulation pédagogique. Il calcule les paramètres principaux et génère plusieurs figures indépendantes afin de visualiser le PWM, le condensateur, les courants et la comparaison des pertes BJT/MOSFET.

Le script ci-dessous génère directement les sept figures décrites dans ce document et les sauvegarde automatiquement en PNG à 300 dpi. Les mêmes fichiers peuvent être insérés dans Word ou publiés sur GitHub.

```
%% AHDS - Analyse pratique de circuits avec MATLAB #02
% NE555 PWM LED Dimmer - comparaison BJT / N-MOSFET
% Ce script effectue les calculs, génère 7 figures et les sauvegarde en PNG.

clear; clc; close all; % Nettoie l'espace de travail et ferme les figures.

%% 1. Paramètres généraux
VCC = 12; % Tension d'alimentation [V].
Rpot = 50e3; % Potentiomètre de réglage [ohm].
Rmin = 1e3; % Résistance minimale R3 [ohm].
Ctim = 100e-9; % Condensateur de temporisation C1 [F].
Rled = 68; % Résistance série de chaque branche LED [ohm].
Vf = 3.0; % Tension directe approximative d'une LED [V].
Nseries = 3; % Nombre de LED en série dans chaque branche.
Nbranches = 4; % Nombre de branches du schéma initial.
VCEsat = 0.30; % Chute de tension BJT en saturation [V].
Rds_on = 20e-3; % RDS(on) d'un MOSFET d'exemple [ohm].
Vout555 = 11; % Niveau haut approximatif de la sortie NE555 [V].
VBE = 0.8; % Tension base-émetteur approximative [V].
Rb = 1e3; % Résistance de base R2 corrigée à 1 kOhm.

%% 2. Courant de base du BJT
Ib = (Vout555 - VBE) / Rb; % Calcule le courant de base.
fprintf('Courant de base BJT : %.2f mA\n', Ib*1e3);

%% 3. Fréquence nominale du PWM
% Avec les deux diodes, la somme des chemins de charge et de décharge
% reste approximativement égale à Rpot + 2*Rmin.
Tnom = 0.693 * (Rpot + 2*Rmin) * Ctim; % Période nominale [s].
fPWM = 1 / Tnom; % Fréquence nominale [Hz].
fprintf('Frequence PWM approximative : %.1f Hz\n', fPWM);

%% 4. Courant LED à l'état ON avec BJT
Ibranch_on = max((VCC - Nseries*Vf - VCEsat) / Rled, 0); % Courant d'une branche [A].
Itotal_on = Nbranches * Ibranch_on; % Courant total des quatre branches [A].
fprintf('Courant ON d'une branche : %.1f mA\n', Ibranch_on*1e3);
fprintf('Courant ON total : %.1f mA\n', Itotal_on*1e3);

%% 5. Figure 1 - PWM pour plusieurs rapports cycliques
duties = [0.10 0.25 0.50 0.75 0.90]; % Rapports cycliques étudiés.
t = linspace(0, 4*Tnom, 4000); % Base temporelle sur quatre périodes.

figure('Name','PWM Duty Cycles'); % Crée la première figure.
hold on; % Permet de superposer plusieurs courbes.
offset = 0; % Décalage vertical pour faciliter la lecture.

for k = 1:length(duties) % Parcourt les cinq rapports cycliques.
    D = duties(k); % Sélectionne le rapport cyclique courant.
    pwm = mod(t,Tnom) < D*Tnom; % Génère le signal logique PWM.
    plot(t*1e3, double(pwm)+offset, ...
        'LineWidth',1.4, ...
        'DisplayName',sprintf('%d %%',round(D*100))); % Trace le PWM.
    offset = offset + 1.4; % Décale la courbe suivante.
end

grid on; % Active la grille.
xlabel('Temps [ms]'); % Nomme l'axe X.
ylabel('Niveau logique + décalage'); % Nomme l'axe Y.
title('NE555 PWM - Influence du rapport cyclique'); % Ajoute le titre.
legend('Location','best'); % Affiche la légende.
exportgraphics(gcf,'01_PWM_duty_cycles.png','Resolution',300); % Sauvegarde le PNG.

%% 6. Figure 2 - Charge et décharge du condensateur C1
alpha = 0.50; % Position du potentiomètre : 50 %.
Rcharge = Rmin + alpha*Rpot; % Résistance équivalente de charge.
Rdischarge = Rmin + (1-alpha)*Rpot; % Résistance équivalente de décharge.
ton = 0.693*Rcharge*Ctim; % Temps ON approximatif.
toff = 0.693*Rdischarge*Ctim; % Temps OFF approximatif.
T = ton + toff; % Période correspondante.
t2 = linspace(0,5*T,5000); % Base temporelle sur cinq périodes.
```

```

phase = mod(t2,T); % Position temporelle dans chaque période.
Vlow = VCC/3; % Seuil bas du NE555.
Vhigh = 2*VCC/3; % Seuil haut du NE555.
Vc = zeros(size(t2)); % Préalloue le vecteur de tension C1.

for k = 1:length(t2) % Calcule la tension du condensateur.
    if phase(k) < ton % Teste si C1 est en phase de charge.
        Vc(k) = VCC-(VCC-Vlow)*exp(-phase(k)/(Rcharge*Ctim)); % Charge exponentielle.
    else % Sinon, C1 est en phase de décharge.
        td = phase(k)-ton; % Calcule le temps depuis le début de la décharge.
        Vc(k) = Vhigh*exp(-td/(Rdischarge*Ctim)); % Décharge exponentielle.
    end
end

figure('Name','C1 Charge-Decharge'); % Crée la deuxième figure.
plot(t2*1e3,Vc,'LineWidth',1.4); % Trace la tension de C1.
hold on; % Permet d'ajouter les seuils.
yline(Vlow,'--','1/3 VCC'); % Trace le seuil bas.
yline(Vhigh,'--','2/3 VCC'); % Trace le seuil haut.
grid on; % Active la grille.
xlabel('Temps [ms]'); % Nomme l'axe X.
ylabel('Tension V_C [V]'); % Nomme l'axe Y.
title('Charge et decharge du condensateur C1'); % Ajoute le titre.
exportgraphics(gcf,'02_C1_charge_decharge.png','Resolution',300); % Sauvegarde le PNG.

%% 7. Figure 3 - Courants instantanés des LED
pwm50 = phase < ton; % Génère le PWM correspondant à alpha = 50 %.
Iled_t = Ibranch_on*doubling(pwm50); % Courant instantané d'une branche.
Itotal_t = Itotal_on*doubling(pwm50); % Courant instantané total.

figure('Name','LED Current'); % Crée la troisième figure.
plot(t2*1e3,Iled_t*1e3,'LineWidth',1.4); % Trace le courant d'une branche.
hold on; % Permet d'ajouter le courant total.
plot(t2*1e3,Itotal_t*1e3,'LineWidth',1.4); % Trace le courant des quatre branches.
grid on; % Active la grille.
xlabel('Temps [ms]'); % Nomme l'axe X.
ylabel('Courant [mA]'); % Nomme l'axe Y.
title('Courant instantane des LED - PWM a 50 %'); % Ajoute le titre.
legend('Une branche','4 branches','Location','best'); % Ajoute la légende.
exportgraphics(gcf,'03_LED_current_waveform.png','Resolution',300); % Sauvegarde le PNG.

%% 8. Figure 4 - Courant moyen en fonction du duty cycle
Dvec = linspace(0,1,101); % Crée un balayage de 0 à 100 %.
Iavg_branch = Ibranch_on.*Dvec; % Calcule le courant moyen d'une branche.
Iavg_total = Itotal_on.*Dvec; % Calcule le courant moyen total.

figure('Name','Average Current'); % Crée la quatrième figure.
plot(Dvec*100,Iavg_branch*1e3,'LineWidth',1.4); % Trace le courant moyen d'une branche.
hold on; % Permet d'ajouter le courant total.
plot(Dvec*100,Iavg_total*1e3,'LineWidth',1.4); % Trace le courant moyen total.
grid on; % Active la grille.
xlabel('Rapport cyclique [%]'); % Nomme l'axe X.
ylabel('Courant moyen [mA]'); % Nomme l'axe Y.
title('Courant moyen des LED en fonction du duty cycle'); % Ajoute le titre.
legend('Une branche','4 branches','Location','best'); % Ajoute la légende.
exportgraphics(gcf,'04_Average_current_vs_duty.png','Resolution',300); % Sauvegarde le PNG.

%% 9. Figure 5 - Comparaison des pertes BJT / MOSFET
Ivec = linspace(0,5,300); % Balaye le courant de charge de 0 à 5 A.
Dcompare = 0.80; % Utilise 80 % de duty cycle pour la comparaison.
Pbjt = VCEsat.*Ivec.*Dcompare; % Calcule les pertes de conduction du BJT.
Pmos = (Ivec.^2).*Rds_on.*Dcompare; % Calcule les pertes de conduction du MOSFET.

figure('Name','BJT vs MOSFET'); % Crée la cinquième figure.
plot(Ivec,Pbjt,'LineWidth',1.4); % Trace les pertes du BJT.
hold on; % Permet d'ajouter la courbe MOSFET.
plot(Ivec,Pmos,'LineWidth',1.4); % Trace les pertes du MOSFET.
grid on; % Active la grille.
xlabel('Courant de charge [A]'); % Nomme l'axe X.
ylabel('Pertes de conduction [W]'); % Nomme l'axe Y.
title('Comparaison des pertes - BJT vs N-MOSFET'); % Ajoute le titre.
legend('BJT','N-MOSFET','Location','northwest'); % Ajoute la légende.
exportgraphics(gcf,'05_BJT_vs_MOSFET_losses.png','Resolution',300); % Sauvegarde le PNG.

```

```

%% 10. Figure 6 - Courant total en fonction du nombre de LED
Nled = 3:3:300; % Fait varier le nombre total de LED de 3 à 300.
Nstrings = Nled/3; % Calcule le nombre de branches de trois LED.
Istring = 20e-3; % Fixe 20 mA par branche pour cette étude d'échelle.
Iload = Nstrings.*Istring; % Calcule le courant total à 100 % duty cycle.

figure('Name','LED Count Current'); % Crée la sixième figure.
plot(Nled,Iload,'LineWidth',1.4); % Trace le courant total.
grid on; % Active la grille.
xlabel('Nombre total de LED'); % Nomme l'axe X.
ylabel('Courant total a 100 % [A]'); % Nomme l'axe Y.
title('Courant total en fonction du nombre de LED'); % Ajoute le titre.
exportgraphics(gcf,'06_Total_current_vs_LED_count.png','Resolution',300); % Sauvegarde le PNG.

%% 11. Figure 7 - Pertes BJT / MOSFET en fonction du nombre de LED
Pbjt_led = VCEsat.*Iload; % Calcule les pertes BJT à 100 % duty cycle.
Pmos_led = (Iload.^2).*Rds_on; % Calcule les pertes MOSFET à 100 % duty cycle.

figure('Name','Losses vs LED Count'); % Crée la septième figure.
plot(Nled,Pbjt_led,'LineWidth',1.4); % Trace les pertes BJT.
hold on; % Permet d'ajouter les pertes MOSFET.
plot(Nled,Pmos_led,'LineWidth',1.4); % Trace les pertes MOSFET.
grid on; % Active la grille.
xlabel('Nombre total de LED'); % Nomme l'axe X.
ylabel('Pertes de conduction [W]'); % Nomme l'axe Y.
title('Pertes BJT / MOSFET en fonction du nombre de LED'); % Ajoute le titre.
legend('BJT','N-MOSFET','Location','northwest'); % Ajoute la légende.
exportgraphics(gcf,'07_Losses_vs_LED_count.png','Resolution',300); % Sauvegarde le PNG.

%% 12. Fin du calcul
fprintf('\nSimulation terminee.\n'); % Confirme la fin du script.
fprintf('Les 7 graphiques ont ete sauvegardes au format PNG.\n'); % Confirme la sauvegarde.

```

13. Interprétation des résultats

Les figures permettent de relier directement les paramètres électriques au comportement du circuit. Lorsque le rapport cyclique augmente, le courant instantané ON ne change pas dans le modèle idéal, mais le courant moyen et la puissance moyenne augmentent. La comparaison BJT/MOSFET montre également que les pertes ne suivent pas la même loi : elles sont approximativement proportionnelles au courant pour le BJT, alors qu'elles suivent $I^2R_{DS(on)}$ pour la conduction du MOSFET.

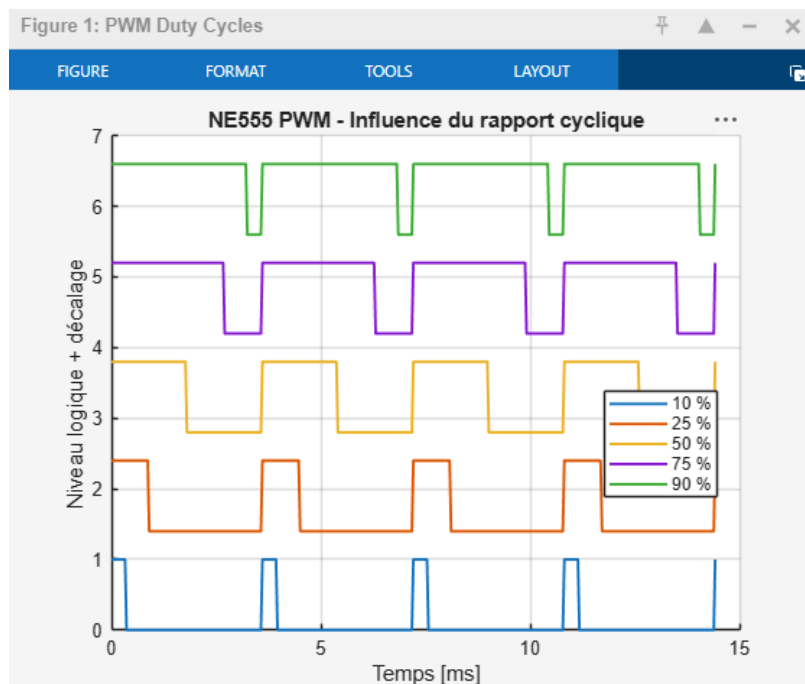


Figure 1 — NE555 PWM : influence du rapport cyclique

Cette figure montre l'évolution du signal PWM pour plusieurs rapports cycliques : 10 %, 25 %, 50 %, 75 % et 90 %. La période du signal reste pratiquement constante, tandis que la durée de l'état haut augmente avec le rapport cyclique. Le NE555 permet ainsi de contrôler la puissance moyenne transmise à la charge sans modifier directement l'amplitude du signal. Ce principe constitue la base de la variation de luminosité des LED par PWM.

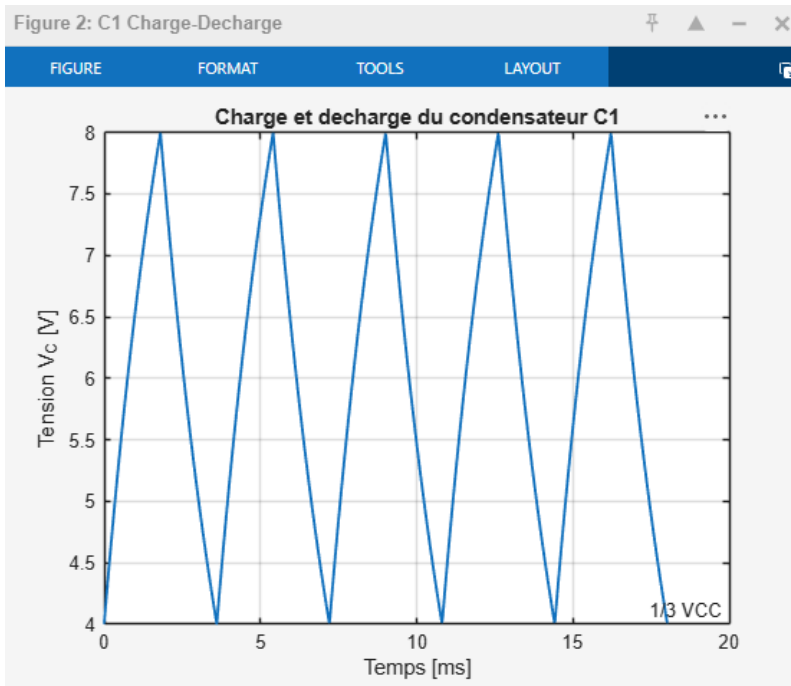


Figure 2 — Charge et décharge du condensateur C1

Cette figure représente l'évolution de la tension aux bornes du condensateur de temporisation C1. La tension varie entre les deux seuils internes du NE555, soit approximativement $1/3 V_{CC}$ et $2/3 V_{CC}$, correspondant ici à 4 V et 8 V. Lorsque l'un des seuils est atteint, l'état interne du NE555 change et le condensateur passe de la charge à la décharge, ou inversement. Ce cycle détermine directement la période et le rapport cyclique du signal PWM.

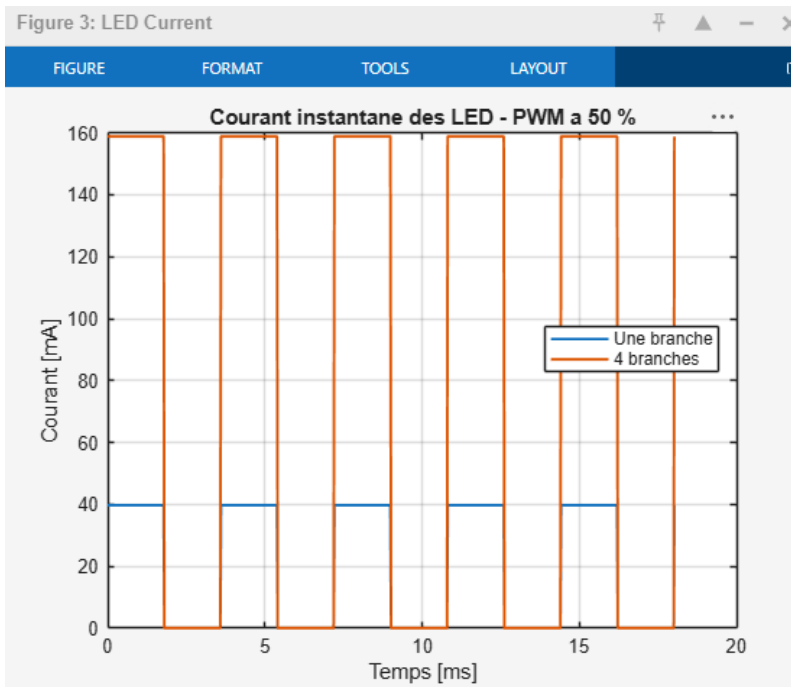


Figure 3 — Courant instantané des LED à 50 % de PWM

Le courant dans les LED suit directement l'état du signal PWM : il circule pendant la phase ON et devient pratiquement nul pendant la phase OFF. La simulation montre simultanément le courant d'une branche et le courant total des quatre branches. À 50 % de rapport cyclique, la valeur instantanée du courant ON reste inchangée, mais le courant ne circule que pendant environ la moitié de chaque période. C'est cette commutation rapide qui permet de réduire la luminosité moyenne perçue.

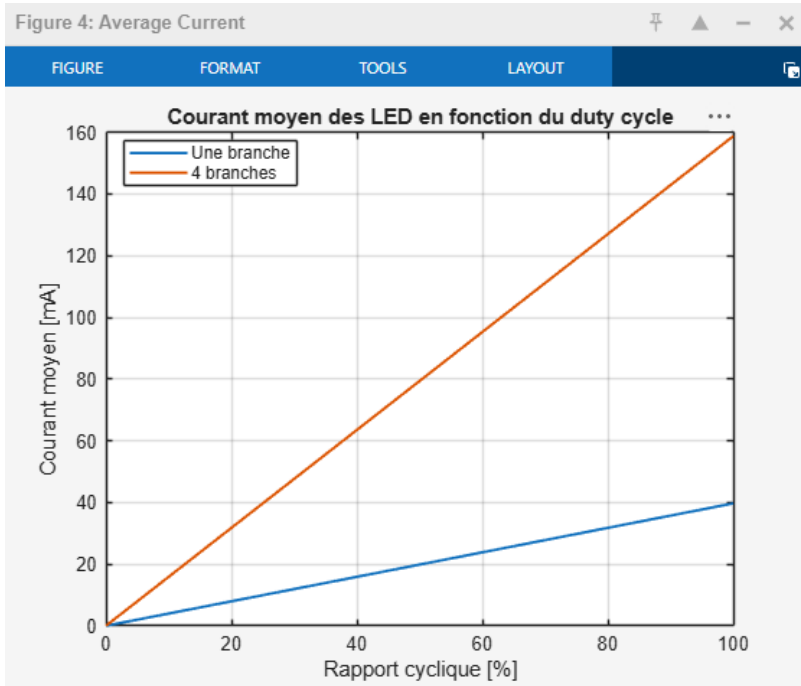


Figure 4 — Courant moyen en fonction du rapport cyclique

Le courant moyen augmente pratiquement linéairement avec le rapport cyclique. À 0 %, aucun courant moyen ne traverse les LED, tandis qu'à 100 %, le courant moyen atteint la valeur correspondant à une conduction permanente. La même relation apparaît pour une seule branche et pour l'ensemble des quatre branches. Cette courbe illustre directement la relation approximative $I_{AVG} = D \times I_{ON}$.

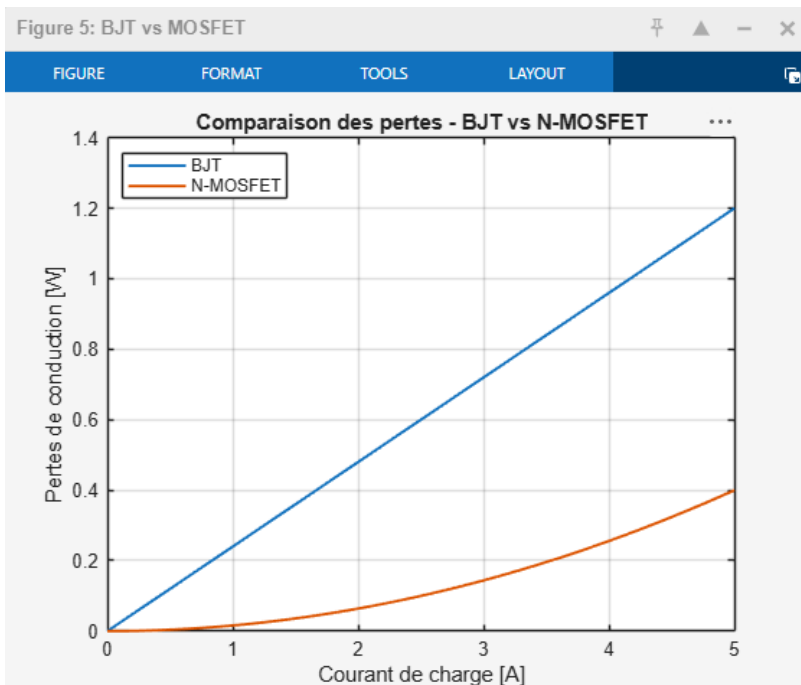


Figure 5 — Comparaison des pertes de conduction : BJT vs N-MOSFET

Cette figure compare les pertes de conduction d'un BJT et d'un MOSFET canal N lorsque le courant de charge augmente. Dans le modèle utilisé, les pertes du BJT augmentent approximativement de façon linéaire avec le courant, tandis que celles du MOSFET suivent la relation $I_D^2 R_{DS(on)}$. Pour la plage de courant étudiée et les paramètres choisis, le MOSFET présente des pertes nettement plus faibles. Cette comparaison montre l'intérêt d'un MOSFET à faible $R_{DS(on)}$ pour commander des charges LED de puissance plus élevée.

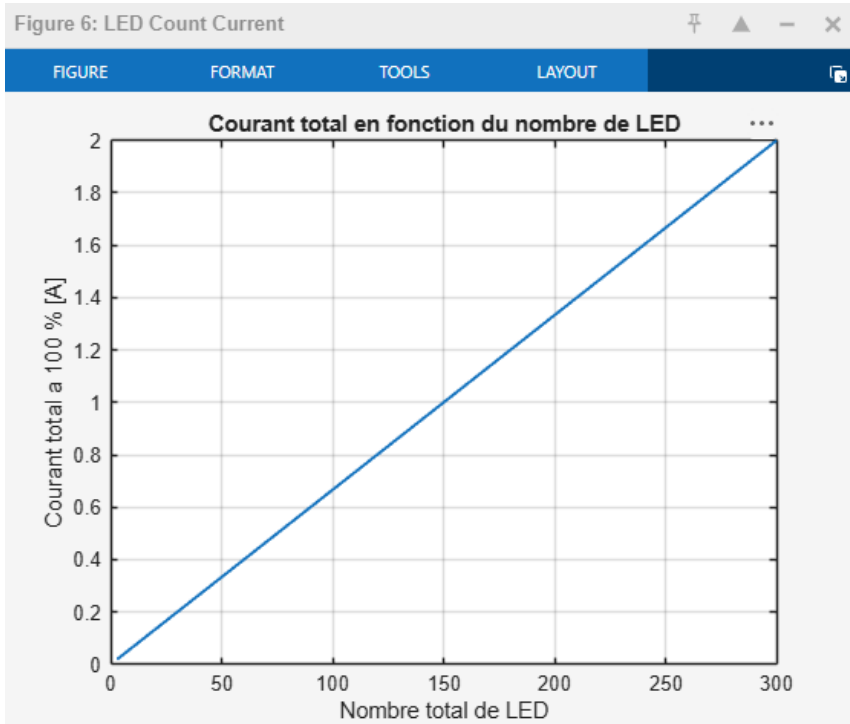


Figure 6 — Courant total en fonction du nombre de LED

Pour cette étude d'échelle, les LED sont regroupées par branches de trois, avec un courant supposé de 20 mA par branche. L'augmentation du nombre de LED entraîne donc une augmentation linéaire du nombre de branches et du courant total demandé à l'alimentation. Avec 300 LED, soit 100 branches, le courant total atteint environ 2 A. Cette représentation permet d'estimer rapidement les besoins en courant lorsqu'on augmente la taille de la charge LED.

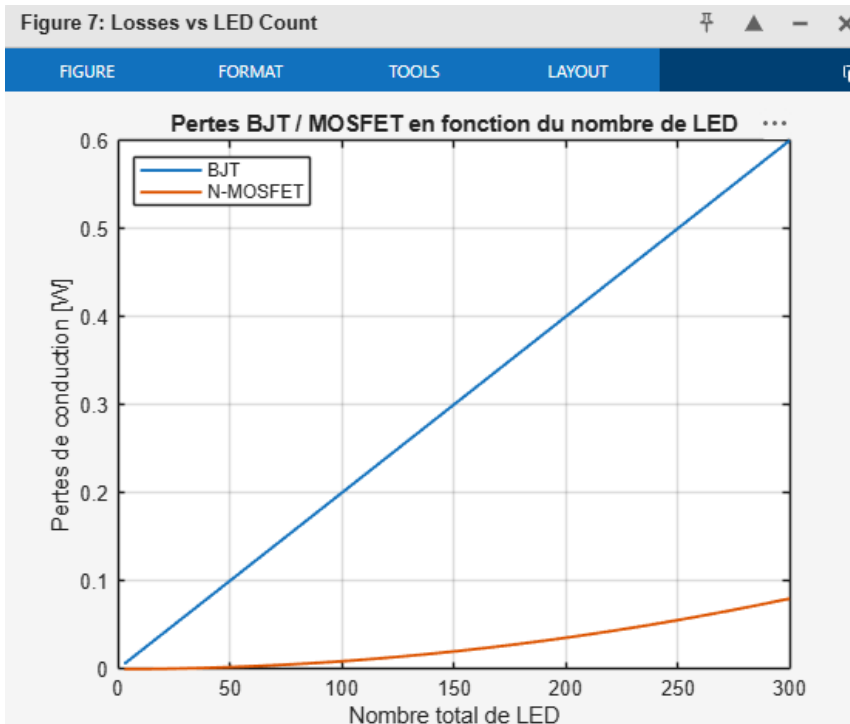


Figure 7 — Pertes BJT / MOSFET en fonction du nombre de LED

Cette figure prolonge l'étude précédente en montrant l'évolution des pertes de conduction lorsque le nombre de LED augmente. Avec l'augmentation du courant total, les pertes du BJT deviennent progressivement plus importantes. Pour les paramètres utilisés dans la simulation, le MOSFET conserve une dissipation nettement inférieure jusqu'à 300 LED. Le graphique met ainsi en évidence l'avantage énergétique du MOSFET pour une commande PWM de puissance plus élevée.

Le modèle peut ensuite être enrichi avec les caractéristiques d'un composant réel : courbe $R_{DS(on)}$ en fonction de V_{GS} et de la température, $V_{CE(sat)}$ du BJT, tension directe réelle des LED, résistance thermique, pertes de commutation et tolérances des composants.

14. Limites du modèle

- Le modèle ne remplace pas une simulation SPICE détaillée ni la validation sur un prototype réel.
- La tension de sortie du NE555 est simplifiée et dépend en réalité du composant et du courant de sortie.
- V_f des LED varie avec le courant et la température.

- VCE(sat) du BJT n'est pas constant et dépend du courant et du courant de base.
- RDS(on) du MOSFET dépend de VGS et augmente généralement avec la température.
- Les pertes de commutation et les capacités parasites peuvent devenir importantes lorsque la fréquence augmente.
- Pour de nombreuses branches LED, l'alimentation, les pistes PCB, les connecteurs et la dissipation thermique doivent être dimensionnés.

15. Conclusion

Ce deuxième exemple montre comment MATLAB peut être utilisé non seulement pour tracer une forme d'onde, mais aussi pour étudier un circuit complet : temporisation du NE555, PWM, courant de charge, puissance, influence du rapport cyclique et comparaison de deux technologies de commutation. L'intérêt principal est de pouvoir modifier rapidement les paramètres et visualiser immédiatement les conséquences électriques.

Une évolution naturelle de cette étude consiste à remplacer les valeurs génériques par les données d'un BJT et d'un MOSFET réels, puis à comparer les résultats MATLAB à une simulation réelle.

AHDS - Analyse pratique de circuits avec MATLAB

Pour toute question ou commentaire : ahds@engineer.com

GitHub : <https://github.com/Anfiska2023>

Exécuter cette simulation avec MATLAB Online

*Il n'est pas nécessaire d'installer MATLAB sur un ordinateur pour exécuter cette simulation. **MATLAB Online** permet d'utiliser MATLAB directement à partir d'un navigateur Web, sans téléchargement ni installation du logiciel. Il suffit de créer gratuitement un compte MathWorks, puis d'ouvrir MATLAB Online.*

*Le programme présenté dans ce document peut être enregistré sous la forme d'un fichier MATLAB portant l'extension **.m**. Ce fichier peut ensuite être téléversé directement dans MATLAB Online ou placé dans MATLAB Drive. Il suffit alors de l'ouvrir dans l'éditeur MATLAB et de sélectionner **Run** pour exécuter la simulation. MATLAB effectue automatiquement les calculs, génère les sept graphiques présentés dans cette étude et les sauvegarde également sous forme de fichiers PNG.*

*La version **MATLAB Online (basic)** est actuellement disponible gratuitement et offre jusqu'à **20 heures d'utilisation par mois**, ainsi qu'un accès à une sélection de produits MATLAB couramment utilisés. Ces limitations peuvent évoluer; il est donc recommandé de consulter les conditions actuelles de MathWorks. Pour les calculs et les graphiques utilisés dans cette étude, aucune installation locale de MATLAB n'est nécessaire.*

Important : cette étude est une simulation mathématique. Aucune carte électronique réelle n'est nécessaire pour exécuter le programme. Le circuit présenté sert de modèle pour analyser le fonctionnement du NE555, le rapport cyclique PWM, le courant des LED et les pertes de conduction du BJT et du MOSFET. Les résultats obtenus sont basés sur les paramètres et les modèles définis dans le programme MATLAB;

Accès à MATLAB Online :

[MATLAB Online — MathWorks](#)